

Prompt templates and reusable patterns

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN

Session 8 of 12 · English notes

After stop sequences and JSON fences, Vijay treated prompts as **templates**: a stable instruction string with holes, filled by application variables (interpolation). The running example was bilingual leave mail — French employee, British manager — without wiring real SMTP. The same template rendered two directions of translation by swapping `style` and `text`.

Interpolation is just substitution

Python triple-quoted strings held multi-line prompts. Placeholders such as `{style}` and `{text}` (or a custom `render(template, **vars)`) injected values at runtime. That is not a Claude-only miracle; it is software. Claude-specific advice was **how you mark the injected blob** so the model does not mix data with instructions.

Separate data from instructions

Anti-pattern: “Write a one-page decision report... here are 20 pages of sales records: `{sales}` ... then follow these steps...” with the records dumped mid-sentence. Better: wrap the records in **XML-style tags you invent** (`<sales_records>...</sales_records>`) or in triple backticks, and say explicitly that only that region is data. Tags are not a reserved Anthropic vocabulary — you name them. The point is bifurcation: this is evidence, this is the task.

The leave-mail template

1. Reusable skeleton: “Translate the text delimited by triple backticks into the style `{style}`.”
2. Employee path: `style` = British English, calm and respectful; `text` = French leave request. Output is English the manager can act on.
3. Manager path: same skeleton; `style` = polite French; `text` = English approval/rejection.
4. SMTP, if you ever add it, only fills `text`; it does not rewrite the skeleton.

Helpers from module 07 (`add_user_message`, `chat` with `max_tokens` and stop sequences) wrapped the rendered string. Templates can also inject few-shot examples or chain-of-thought sketches when those slots are part of the skeleton.

Classic prompting patterns as injectable content

Pattern	What you inject	Watch-out
Zero-shot	Task only	Fine when the task is familiar to the model
Few-shot	Worked input/output pairs	Examples steal context window; keep them short

Pattern	What you inject	Watch-out
Chain of thought	“Show steps” or worked maths	Leaks reasoning to the user; extended thinking (M09) is the productised alternative
XML / fences	Named regions	Be consistent or the model ignores the contract

Why templates exist in applications

Prompts change with every email body. If you hard-code French in the instruction, you cannot reuse the skill for German. Templates turn prompting into configuration. They also make evaluation possible: you store (template + variables) → output as a dataset (next chapter).

DO NOT CONFUSE INTERPOLATION WITH PROMPT CACHE

Substituting variables creates a *new* prompt string. Cache hits require a stable prefix (module 09). Put the huge invariant instructions first; put the tiny text after the cache breakpoint if you use caching.

Lab plumbing (skip in the exam essay, know in the MCQ)

Install, dotenv, client, add_user / add_assistant, chat() — same first cells as 17 August. The new idea is render, not a new SDK verb. Triple backticks in the instruction must match whatever you actually concatenate around text.

Study drill

Given a sales-drop report prompt, rewrite it so records sit in <data> and numbered steps sit outside. That single move is what Vijay wanted you to remember.

Official documentation

- **Prompt engineering overview**

<https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>

XML tags, examples, and clear instructions.

- **Messages API**

<https://docs.claude.com/en/api/messages>

Where the rendered user string actually goes.

- **Prompt caching**

<https://docs.claude.com/en/docs/build-with-claude/prompt-caching>

Keep static template heads cacheable.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic’s current docs over session anecdotes. Not official exam material.